

HY-463 – Συστήματα Ανάκτησης Πληροφοριών
Project: Φάση Α
Κοσμίδης Νικόλαος csd-5118
Μάνος Αλέξανδρος csd-5136

23 Απριλίου 2026

Εκπόνηση Εργασίας

Για να ναι δίκαιη η κατανομή της εργασίας και για να υπάρχει μια επαφή με το πρότζεκτ και από τα δύο μέλη της ομάδας μας, την Α φάση την χωρίσαμε ως εξής:

- B1-B4 (σύνολο 13 μονάδες) → Αλέξανδρος
- B5-B6 (σύνολο 17 μονάδες) → Νίκος

Υλοποίηση

Για την υλοποίηση της διαδικασίας ευρετηρίασης, αντί να τα βάλουμε όλα σε μία κλάση (όπως κάναμε στην αρχή) το σπάσαμε στις εξής κλάσεις:

- DocInfo
 - Έχει τρία πεδία: το Document ID, το relative path και το document norm
- DocumentParser
 - Διαβάζει τα αρχεία nxm1 αρχεία που περνάμε σαν όρισμα στην μέθοδο countWords, τα κάνει parse (Title, Abstract..) και χρησιμοποιεί τον TextAnalyzer για να εξάγει το "καθαρό" κείμενο.
- Indexer
 - Παίρνει τις λέξεις μετά την επεξεργασία τους, της αποθηκεύει σε μία δομή TreeMap, γράφει τα δεδομένα στον δίσκο, δημιουργεί τα Partial Indexes files και τα συγχωνεύει.

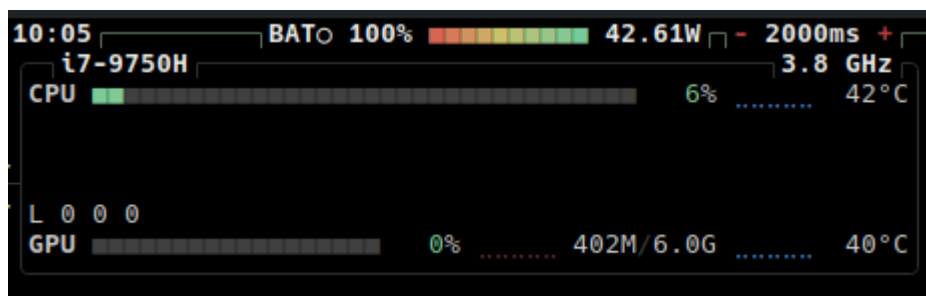
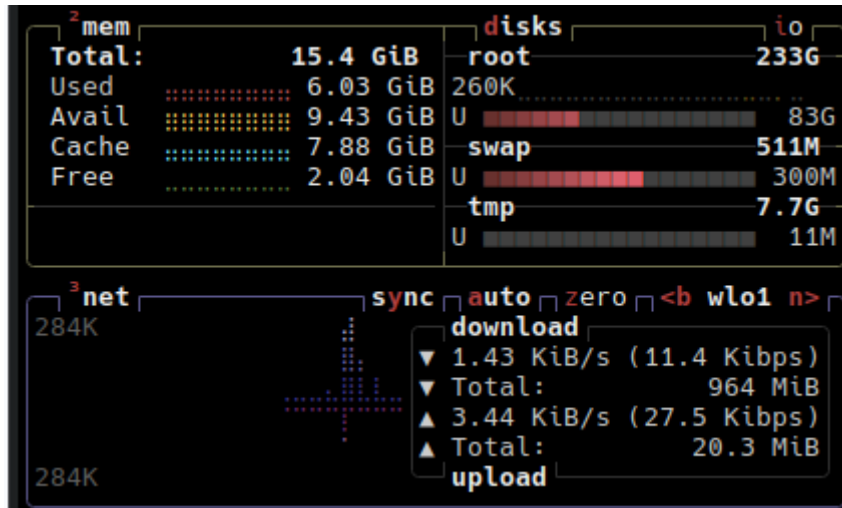
- Posting
 - Έχει τρία πεδία: το ID του εγγράφου, το TF (Term Frequency) της λέξης και μία λίστα με τα position της λέξης στο κείμενο.
- TermInfo
 - Αποθηκεύουμε τις πληροφορίες για κάθε unique word: το DF (Document Frequency), το CF (Concept Frequency) και τα postings της λέξης για όλα τα έγγραφα.
- TextAnalyzer
 - Δημιουργεί το Regex, τα StopWords και το Stemming για τον “καθαρισμό” του κειμένου/των λέξεων, επιστρέφοντας μόνο τα tokens.
- WordCounter
 - Η main κλάση όπου σαρώνει τους φακέλους και καλεί τις κατάλληλες μεθόδους των άλλων κλάσεων (Controller)

Για απλό testing έχουμε το DemoMode όπου τρέχουμε το σύστημά μας για ένα αρχείο μόνο. Έπειτα έχουμε δύο μεταβλητές στην main την collectionDir, όπου αντιστοιχεί στο Mini Collection και την fullCollectionDir, όπου αντιστοιχεί στο Medical Collection.

Στην αρχή, επειδή τρέχαμε μόνο το Mini Collection, το partial indexing και merging δεν χρειαζόταν. Όμως όταν τρέξαμε το Medical Collection κράσαρε. Οπότε το υλοποιήσαμε και με partial indexing και merging. Έτσι στον φάκελο CollectionIndex αποθηκεύουμε τα αρχεία DocumentsFile.txt, PostingFile.bin (είναι σε binary), το VocabularyFile.txt και τέλος στον υποφάκελο temp, τα κομμάτια (partial indexing) των αρχείων PostingFile.bin και VocabularyFile.txt, όπου με threshold 10 βγήκαν περίπου 237 parts για το Mini Collection και 10788 parts για το Medical Collection.

Αποτελέσματα

Τα παρακάτω αποτελέσματα είναι από την εκτέλεση του project σε λάπτοπ, με linux λογισμικό (γιατί στα windows δεν είχαμε περιθώριο να δώσουμε ram 😞) Με επεξεργαστή i7 9th Gen, Maximum Heap Size 6144 (6 GB) και κάρτα γραφικών(just in case) 1660 Ti.



Τα αποτελέσματα που είχαμε χωρίς το partial indexing and merging είναι τα εξής:

- Για το Mini Collection έχουμε
 - Total Execution Time: 00:01:744 (minutes:second:millisecond)
- Medical Collection:
 - java.lang.OutOfMemoryError: Java heap space
 - Total Execution Time: 04:41:729 (minutes:second:millisecond)
 - Με άλλα λόγια ξεμείναμε από μνήμη μετά από περίπου 5 λεπτά

Με το partial indexing and merging:

- MiniCollection:
 - Total Execution Time: 00:01:793 (minutes:second:millisecond)
- Medical Collection:
 - Total Execution Time: 47:48:753 (minutes:second:millisecond)

Δοκιμάσαμε και τον πρώτο τρόπο του φροντιστηρίου, που είναι γράφουμε στον δίσκο όταν φτάνουμε το όριο του heap. Οπότε ορίσαμε ένα ασφαλές όριο για το μέγιστο επιτρεπτό usage της μνήμης. Επειδή όμως πειράζαμε το Maximum Heap Size έπρεπε αυτό το όριο να ναι ποσοστό. Έτσι ακολουθώντας το παράδειγμα από το [stackoverflow](https://stackoverflow.com/questions/1844688/how-can-i-read-all-files-in-a-folder-from-java) πετύχαμε τα εξής αποτελέσματα:

- MiniCollection:
 - Total Execution Time: 00:01:947 (minutes:second:millisecond)
- Medical Collection:
 - Total Execution Time: 19:20:259 (minutes:second:millisecond)

Η δεύτερη μέθοδος threshold (στο πόσα αρχεία):

```
88 // https://stackoverflow.com/questions/1844688/how-can-i-read-all-files-in-a-folder-from-java
89 try (Stream<Path> paths = Files.walk(root)) {
90     paths.filter(Files::isRegularFile)
91         .filter(path -> path.toString().endsWith(".nxml"))
92         .forEach(path -> {
93             parser.countWords(path.toFile(), docIdCounter++, collectionPath);
94             currentBatch++;
95
96             if(currentBatch >= threshold)
97             {
98                 System.out.println("Saving partial index " + partID);
99                 indexer.SavePartialIndex(partID++);
100                 indexer.getVocab().clear();
101                 currentBatch = 0;
102             }
103         });
104
105     if (!indexer.getVocab().isEmpty())
106     {
107         indexer.SavePartialIndex(partID++);
108     }
109
110     indexer.MergePartialIndexes(partID - 1);
111     indexer.DocumentsFile();
112
113     System.out.println("Collection processing completed.");
114 } catch (Exception e) {
115     System.err.println("Error while traversing the folder path.");
116 }
```

Η πρώτη και η πιο βέλτιστη μέθοδος:

```
88
89 // https://stackoverflow.com/questions/1844688/how-can-i-read-all-files-in-a-folder-from-java
90 try (Stream<Path> paths = Files.walk(root)) {
91     paths.filter(Files::isRegularFile)
92         .filter( Path path -> path.toString().endsWith(".xml"))
93         .forEach( Path path -> {
94             parser.countWords(path.toFile(), docIdCounter++, collectionPath);
95
96             // dynamic ram check
97             // https://stackoverflow.com/questions/12807797/java-get-available-memory
98
99             Runtime runtime = Runtime.getRuntime();
100             long usedMemory = runtime.totalMemory() - runtime.freeMemory();
101             // Maximum Heap Size --> set to 6144 MiB
102             long maxMemory = runtime.maxMemory();
103             double memoryUsage = (double) usedMemory / maxMemory;
104
105             // if ram usage exceeds 80%, move to disk
106             if (memoryUsage >= ramUsageBound) {
107                 System.out.println("Memory at "
108                     + (memoryUsage * 100) + "%. Saving partial index " + partID);
109
110                 indexer.SavePartialIndex(partID++);
111                 indexer.getVocab().clear();
112
113                 // run garbage collector to reclaim the cleared memory
114                 // https://stackoverflow.com/questions/1481178/how-to-force-garbage-collection-in-java
115                 System.gc();
116             }
117         });
```

Note: πρέπει να [καλέσουμε και τον garbage collector](#) για να καθαρίσει η μνήμη.